ĐẠI HỌC BÁCH KHOA HÀ NỘI
TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG

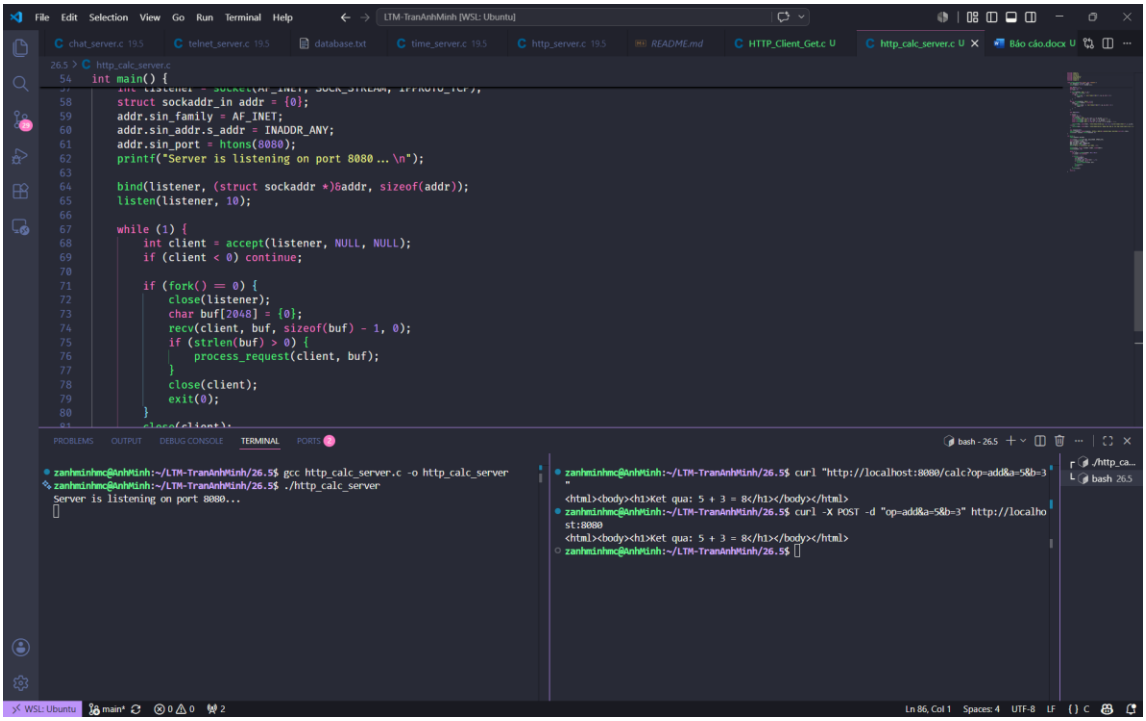
BÁO CÁO BÀI TẬP
LẬP TRÌNH MẠNG

Trần Anh Minh - 20235382

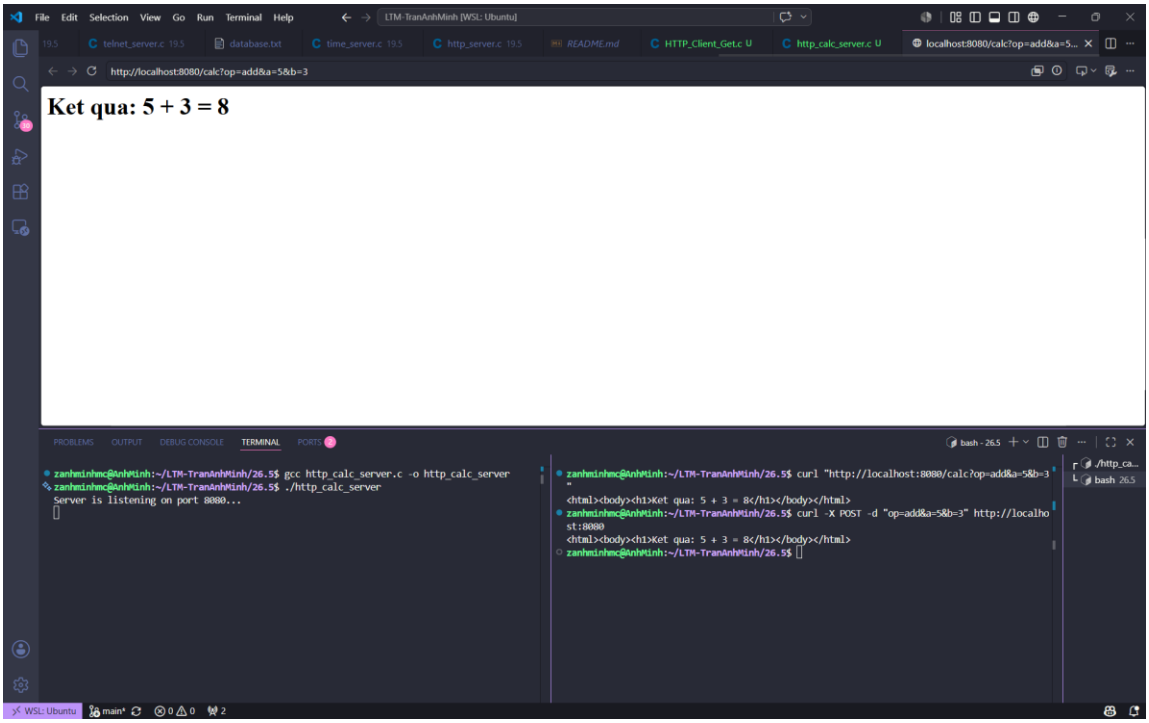
HÀ NỘI, 06/2026

Bài 1 http_calc_server

Ảnh lệnh khởi tạo bên trái, bên phải là get rồi post



Ảnh gọi lệnh bằng link web



Source code

```
      #include <stdio.h>
      #include <stdlib.h>
      #include <string.h>
      #include <sys/socket.h>
      #include <arpa/inet.h>
      #include <unistd.h>
      #include <signal.h>

      void process_request(int client, char *request) {
          char method[16], uri[256], *body;
          sscanf(request, "%s %s", method, uri);

          char op[16] = "";
          double a = 0, b = 0;
          int found = 0;

          if (strcmp(method, "GET") == 0) {
              char *qs = strchr(uri, '?');
              if (qs) {
                  if (sscanf(qs + 1, "op=%[^&]&a=%lf&b=%lf", op, &a, &b) == 3) {
                      found = 1;
                  }
              }
          }
          else if (strcmp(method, "POST") == 0) {
              body = strstr(request, "\r\n\r\n");
              if (body) {
                  if (sscanf(body + 4, "op=%[^&]&a=%lf&b=%lf", op, &a, &b) == 3)
{
                      found = 1;
                  }
```

```
            }
        }

        char html[1024];

        if (found) {
            double res = 0;
            char op_char = '?';
            if (strcmp(op, "add") == 0) { res = a + b; op_char = '+'; }
            else if (strcmp(op, "sub") == 0) { res = a - b; op_char = '-'; }
            else if (strcmp(op, "mul") == 0) { res = a * b; op_char = '*'; }
            else if (strcmp(op, "div") == 0 && b != 0) { res = a / b; op_char =
'/'; }

             snprintf(html, sizeof(html), "<html><body><h1>Ket qua: %g %c %g =
%g</h1></body></html>\n", a, op_char, b, res);
        } else {
            snprintf(html, sizeof(html), "<html><body><h2>Loi: Khong thay tham
so. Vui long truyen tham so op, a, b.</h2></body></html>\n");
        }

        char response[2048];
        snprintf(response, sizeof(response), "HTTP/1.1 200 OK\r\nContent-Type:
text/html\r\n\r\n%s", html);
        send(client, response, strlen(response), 0);
    }

    int main() {
        signal(SIGCHLD, SIG_IGN);

        int listener = socket(AF_INET, SOCK_STREAM, IPPROTO_TCP);
        struct sockaddr_in addr = {0};
        addr.sin_family = AF_INET;
```

```
    addr.sin_addr.s_addr = INADDR_ANY;
    addr.sin_port = htons(8080);
    printf("Server is listening on port 8080...\n");

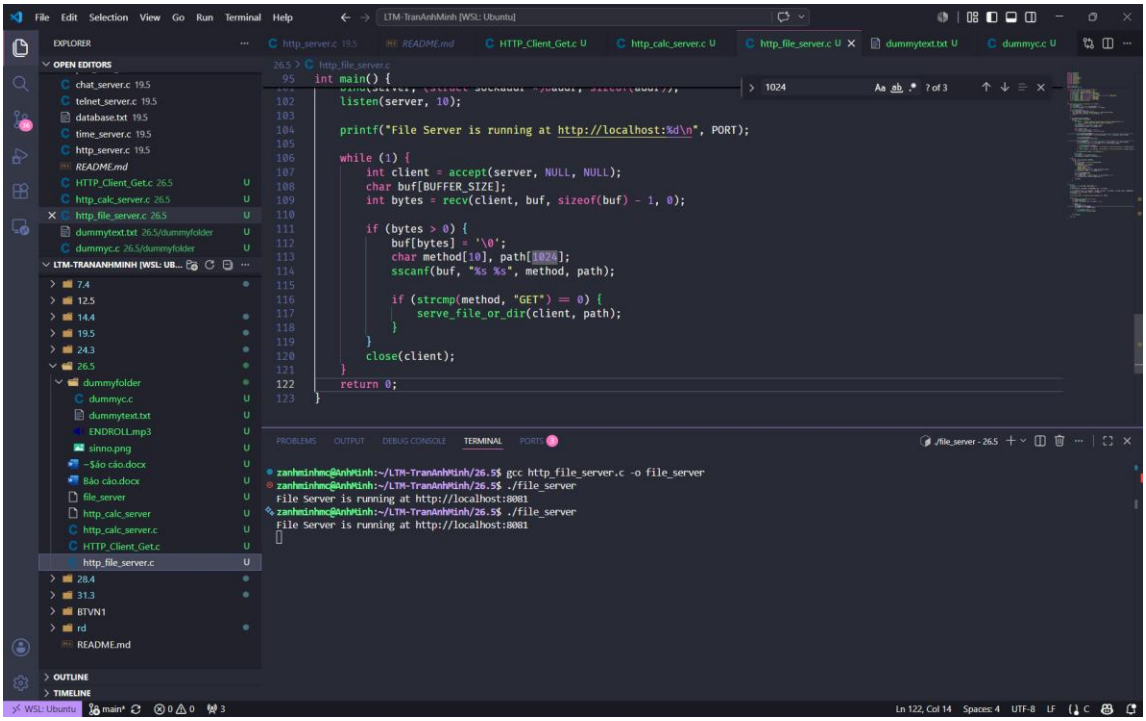
    bind(listener, (struct sockaddr *)&addr, sizeof(addr));
    listen(listener, 10);

    while (1) {
        int client = accept(listener, NULL, NULL);
        if (client < 0) continue;

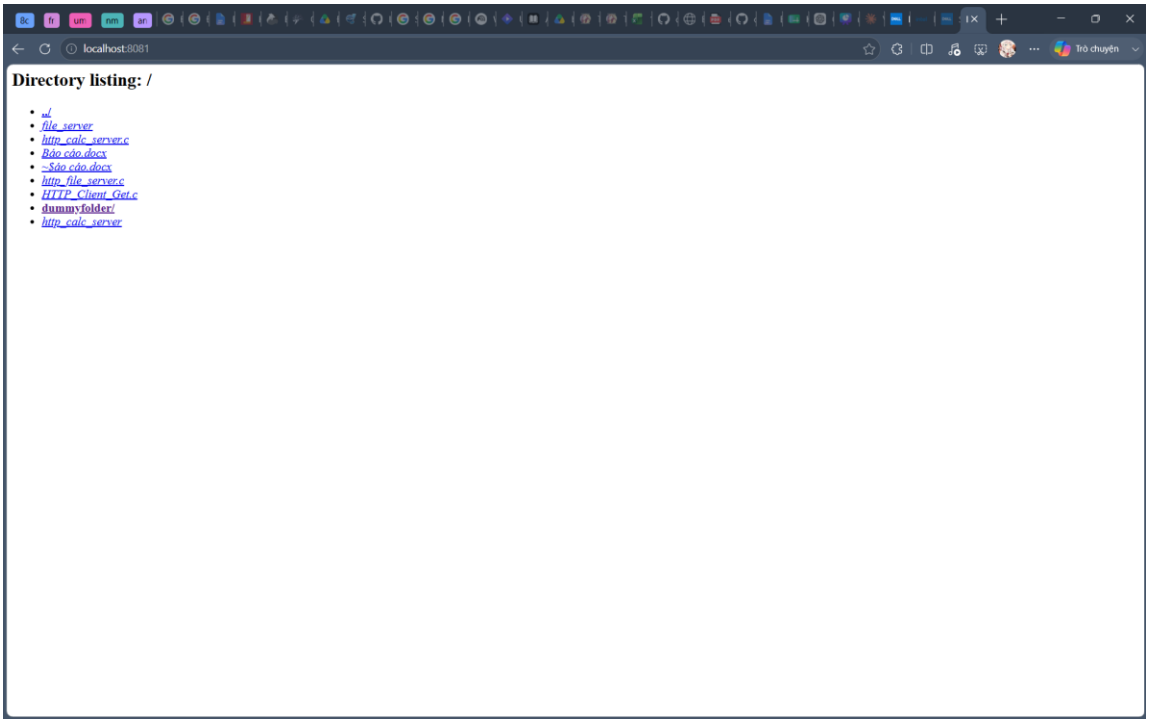
        if (fork() == 0) {
            close(listener);
            char buf[2048] = {0};
            recv(client, buf, sizeof(buf) - 1, 0);
            if (strlen(buf) > 0) {
                process_request(client, buf);
            }
            close(client);
            exit(0);
        }
        close(client);
    }
    return 0;
}
```

Bài 2

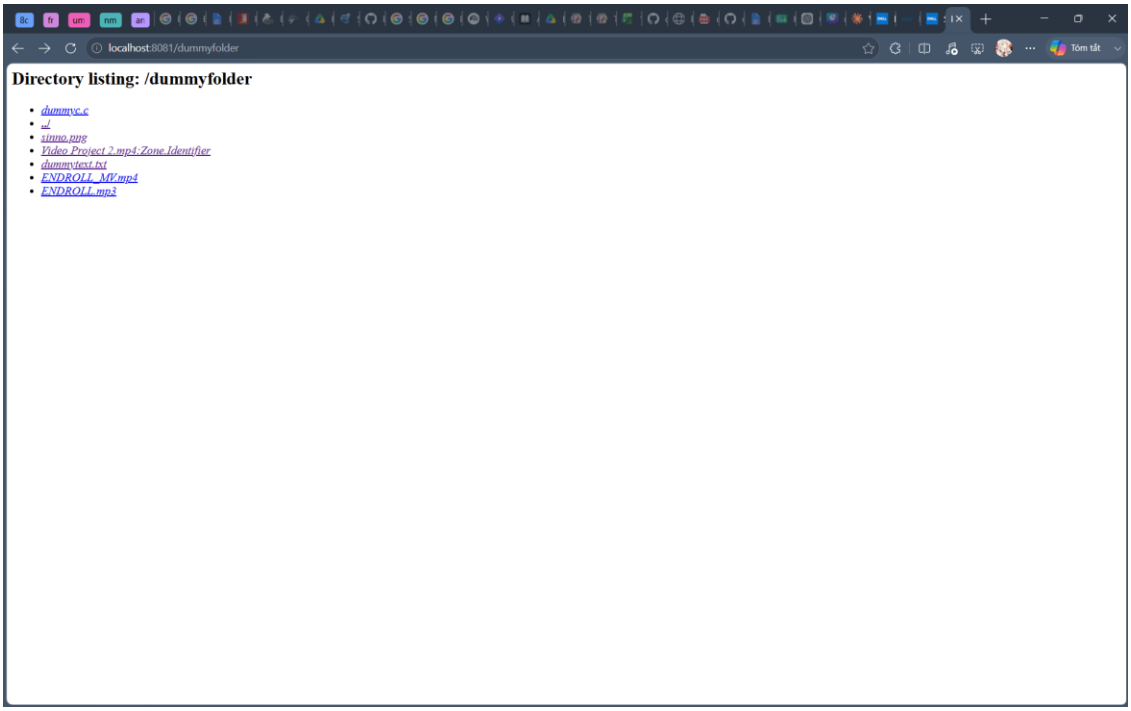
Ảnh khởi tạo và chạy server



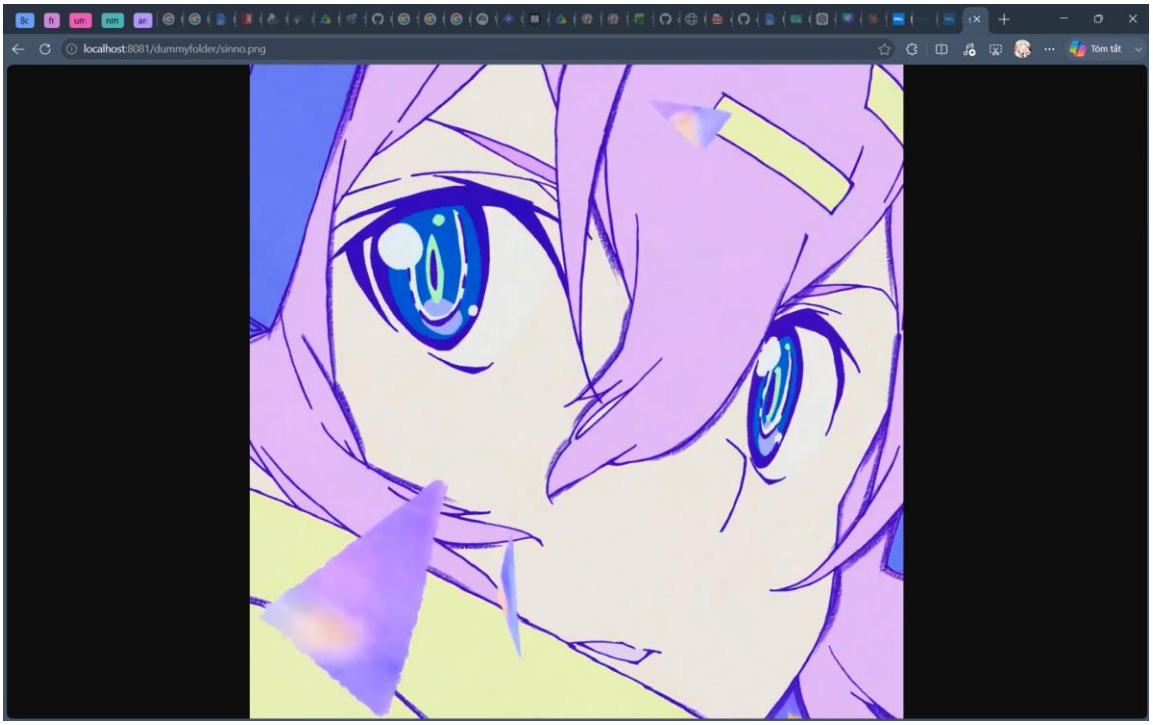
Ảnh hiện khi vào web



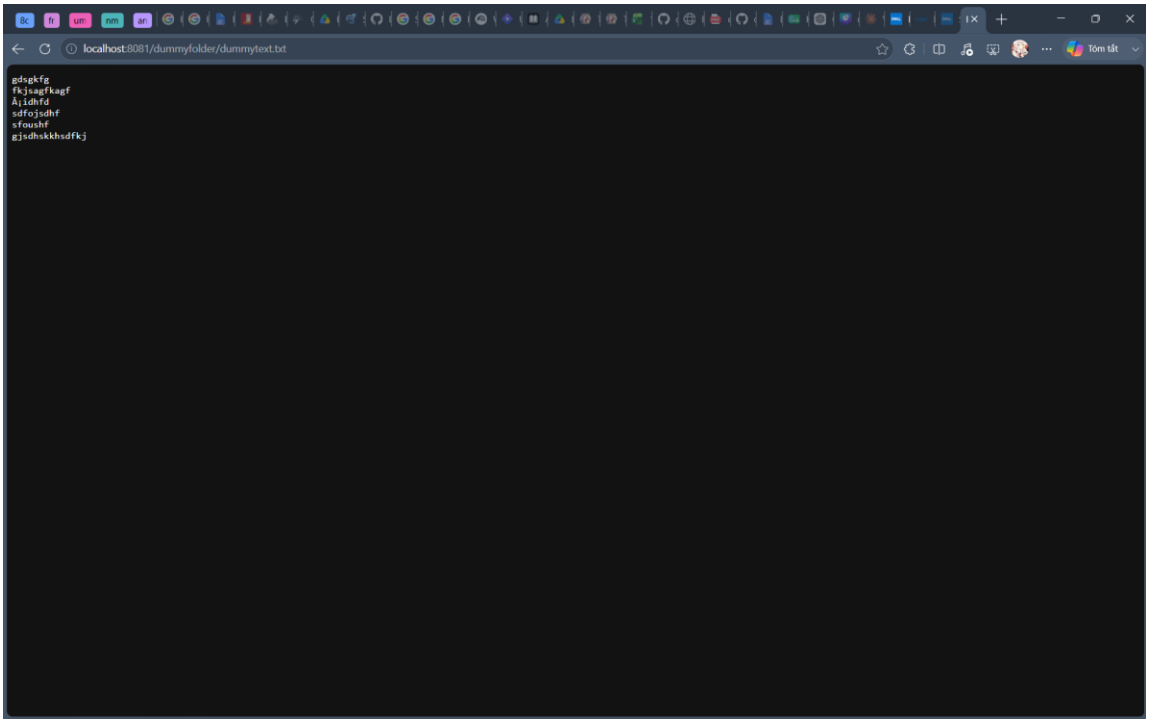
Ảnh hiện các file khi bấm vào folder dummyfolder



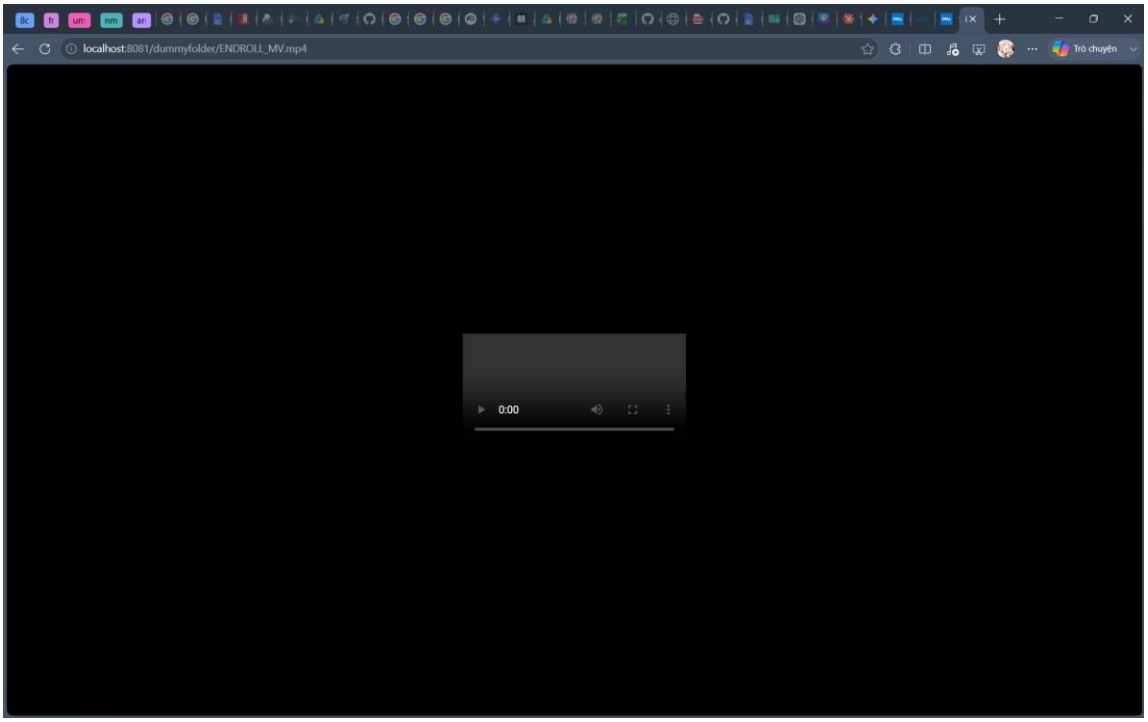
Ảnh khi chọn ảnh



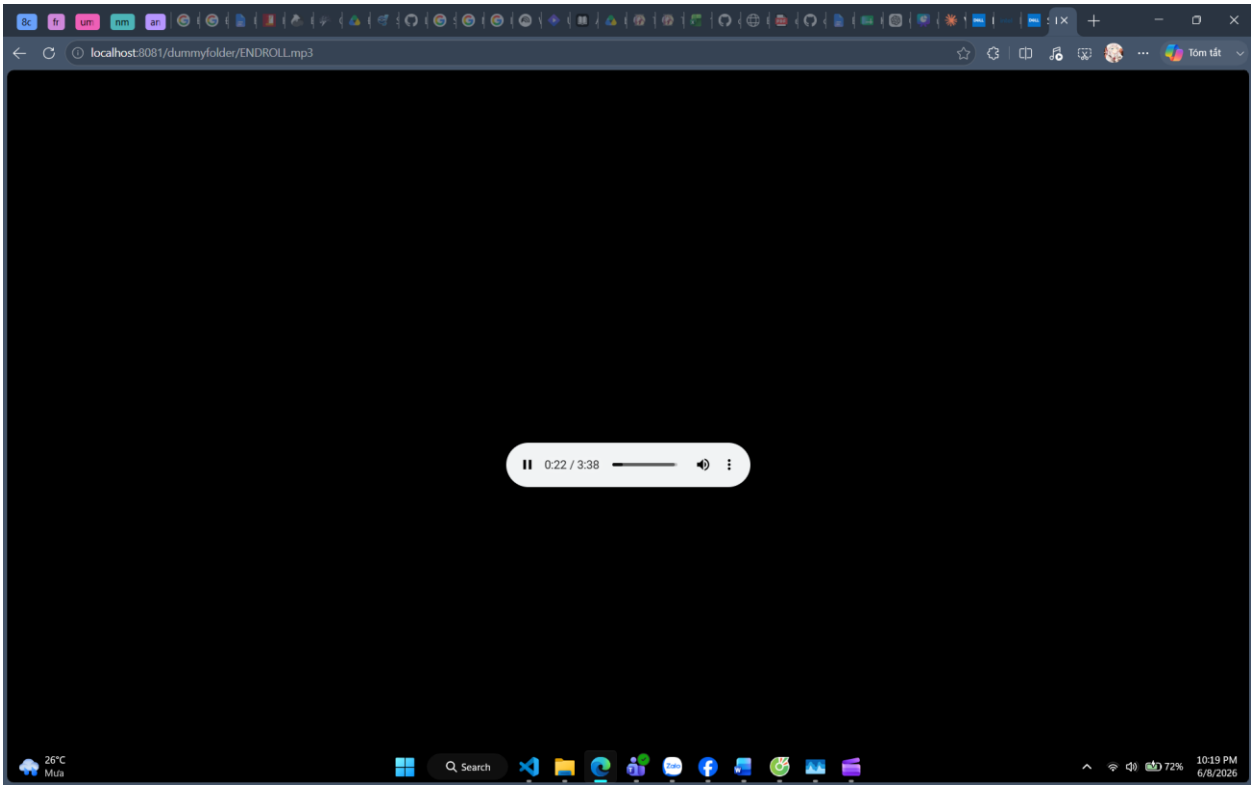
Ảnh khi chọn text



Ảnh khi chọn video



Ảnh khi chọn mp3



Source code

```
      #define _GNU_SOURCE
      ```c
      #define _GNU_SOURCE
      #include <stdio.h>
      #include <stdlib.h>
      #include <string.h>
      #include <sys/socket.h>
      #include <arpa/inet.h>
      #include <unistd.h>
      #include <dirent.h>
      #include <sys/stat.h>
      #include <fcntl.h>

      #define PORT 8081
      #define BUFFER_SIZE 8192

      const char *get_mime_type(const char *filename) {
          const char *ext = strrchr(filename, '.');
          if (!ext) return "application/octet-stream";
          if (strcmp(ext, ".html") == 0) return "text/html";
          if (strcmp(ext, ".txt") == 0) return "text/plain";
          if (strcmp(ext, ".jpg") == 0 || strcmp(ext, ".jpeg") == 0) return
"image/jpeg";
          if (strcmp(ext, ".png") == 0) return "image/png";
          if (strcmp(ext, ".mp4") == 0) return "video/mp4";
          if (strcmp(ext, ".mp3") == 0) return "audio/mpeg";
          return "application/octet-stream";
      }

      void serve_file_or_dir(int client_sock, char *path) {
```
```

```
          char local_path[512];
          if (strcmp(path, "/") == 0) strcpy(local_path, ".");
          else snprintf(local_path, sizeof(local_path), ".%s", path);

          struct stat path_stat;
          if (stat(local_path, &path_stat) != 0) {
              char *not_found = "HTTP/1.1 404 Not Found\r\n\r\n404 File Not Found";
              send(client_sock, not_found, strlen(not_found), 0);
              return;
          }

          if (S_ISDIR(path_stat.st_mode)) {
              DIR *dir = opendir(local_path);
              if (dir) {
                  char header[] = "HTTP/1.1 200 OK\r\nContent-Type: text/html;
charset=utf-8\r\n\r\n"
                                  "<html><body><h2>Directory            listing:
%s</h2><ul>";
                  char buffer[4096];
                  snprintf(buffer, sizeof(buffer), header, path);
                  send(client_sock, buffer, strlen(buffer), 0);

                  struct dirent *entry;
                  while ((entry = readdir(dir)) != NULL) {
                      if (strcmp(entry->d_name, ".") == 0) continue;

                      char full_entry_path[1024];
                      snprintf(full_entry_path, sizeof(full_entry_path), "%s/%s",
local_path, entry->d_name);

                      struct stat entry_stat;
                      stat(full_entry_path, &entry_stat);
```

```
                    char url_path[1024];
                    if (strcmp(path, "/") == 0) snprintf(url_path, sizeof(url_path), "/%s", entry->d_name);
                    else snprintf(url_path, sizeof(url_path), "%s/%s", path, entry->d_name);

                    if (S_ISDIR(entry_stat.st_mode)) {
                        snprintf(buffer, sizeof(buffer), "<li><a href=\"%s\"><b>%s/</b></a></li>", url_path, entry->d_name);
                    } else {
                        snprintf(buffer, sizeof(buffer), "<li><a href=\"%s\"><i>%s</i></a></li>", url_path, entry->d_name);
                    }
                    send(client_sock, buffer, strlen(buffer), 0);
                }
                closedir(dir);
                char *footer = "</ul></body></html>";
                send(client_sock, footer, strlen(footer), 0);
            }
        } else {
            int fd = open(local_path, O_RDONLY);
            if (fd >= 0) {
                const char *mime = get_mime_type(local_path);
                char header[256];
                snprintf(header, sizeof(header),
                    "HTTP/1.1 200 OK\r\n"
                    "Content-Type: %s\r\n"
                    "Content-Length: %ld\r\n"
                    "Connection: close\r\n\r\n", mime, path_stat.st_size);
                send(client_sock, header, strlen(header), 0);

                char file_buf[BUFFER_SIZE];
                ssize_t bytes_read;
```

```
                while ((bytes_read = read(fd, file_buf, sizeof(file_buf))) > 0)
{
                    send(client_sock, file_buf, bytes_read, 0);
                }
                close(fd);
            }
        }
    }

    int main() {
        int server = socket(AF_INET, SOCK_STREAM, 0);
        int opt = 1;
        setsockopt(server, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));

        struct sockaddr_in addr = { .sin_family = AF_INET, .sin_port =
htons(PORT), .sin_addr.s_addr = INADDR_ANY };
        bind(server, (struct sockaddr *)&addr, sizeof(addr));
        listen(server, 10);

        printf("File Server is running at http://localhost:%d\n", PORT);

        while (1) {
            int client = accept(server, NULL, NULL);
            char buf[BUFFER_SIZE];
            int bytes = recv(client, buf, sizeof(buf) - 1, 0);

            if (bytes > 0) {
                buf[bytes] = '\0';
                char method[10], path[1024];
                sscanf(buf, "%s %s", method, path);

                if (strcmp(method, "GET") == 0) {
```

```
                    serve_file_or_dir(client, path);
                }
            }
            close(client);
        }
        return 0;
    }
    ```
```
```